

Top 30 Spring Boot Microservices Interview Questions

1. What is Spring Boot?

Answer:

Spring Boot is a Java framework used to build **standalone, production-ready applications** quickly with minimal configuration.

Key features:

- Auto configuration
 - Embedded server (Tomcat/Jetty)
 - Production ready tools
-

2. What is Microservices Architecture?

Answer:

Microservices architecture is a design where an application is divided into **small independent services**, each performing a specific business function and communicating through **REST APIs**.

3. What is the difference between Monolithic and Microservices Architecture?

Monolithic	Microservices
Single application	Multiple small services
Hard to scale	Easy to scale
Tight coupling	Loose coupling
Single deployment	Independent deployment

4. What are the advantages of Microservices?

- Independent deployment
 - Better scalability
 - Fault isolation
 - Faster development
 - Technology flexibility
-

5. What is Spring Cloud?

Answer:

Spring Cloud provides **tools and frameworks for building distributed systems and microservices** such as service discovery, API gateway, and configuration management.

6. What is Service Discovery?

Answer:

Service discovery allows microservices to **automatically locate each other** using a service registry instead of hardcoding URLs.

Example tool: **Eureka Server**

7. What is Eureka Server?

Answer:

Eureka Server is a **service registry** where all microservices register themselves so that other services can discover and communicate with them.

Components:

- Eureka Server
 - Eureka Client
-

8. What is an API Gateway?

Answer:

API Gateway acts as a **single entry point for all client requests** in a microservices architecture.

Functions:

- Request routing
- Authentication
- Load balancing
- Rate limiting

Example: Spring Cloud Gateway.

9. What is Load Balancing in Microservices?

Answer:

Load balancing distributes incoming requests across multiple instances of a service to improve **performance and reliability**.

Example tools:

- Spring Cloud LoadBalancer
 - Ribbon (older)
-

10. What is Feign Client?

Answer:

Feign Client is a **declarative REST client** used to call other microservices easily by defining an interface.

Example:

```
@FeignClient(name="user-service")
public interface UserClient {
    @GetMapping("/users")
    List<User> getUsers();
}
```

11. What is Circuit Breaker?

Answer:

A circuit breaker prevents system failure when a service is unavailable by **stopping repeated requests to a failing service**.

Example:

- Resilience4j
 - Hystrix (deprecated)
-

12. What is Spring Cloud Config Server?

Answer:

Config Server provides **centralized configuration management** for microservices using a Git repository.

Benefits:

- Central configuration
- Easy updates
- Environment management

13. What is Spring Boot Actuator?

Answer:

Spring Boot Actuator provides **monitoring and management endpoints**.

Examples:

- /health
- /metrics
- /info

14. What is REST API?

Answer:

REST (Representational State Transfer) is an architectural style used for **communication between microservices over HTTP**.

Methods:

- GET
- POST
- PUT
- DELETE

15. What is Dependency Injection?

Answer:

Dependency Injection is a design pattern where **objects are provided by the Spring container instead of creating them manually**.

It improves:

- Loose coupling
- Testability

16. What is the difference between @Component, @Service, and @Repository?

Annotation	Purpose
------------	---------

@Component	Generic Spring component
------------	--------------------------

Annotation	Purpose
@Service	Business logic layer
@Repository	Data access layer

17. What is @RestController?

Answer:

@RestController is used to create **RESTful web services** in Spring Boot.

Example:

```
@RestController
public class UserController {
    @GetMapping("/hello")
    public String hello(){
        return "Hello World";
    }
}
```

18. What is the difference between @Controller and @RestController?

@Controller	@RestController
Used for web applications	Used for REST APIs
Returns view	Returns JSON data

19. What is Docker and why used with Microservices?

Answer:

Docker is a **container platform** used to package applications with their dependencies for easy deployment.

Benefits:

- Consistent environment
 - Easy deployment
 - Lightweight containers
-

20. What is Kubernetes?

Answer:

Kubernetes is a **container orchestration platform** used to manage and scale Docker containers automatically.

Features:

- Auto scaling
 - Load balancing
 - Self healing
-

21. What is Distributed Tracing?**Answer:**

Distributed tracing tracks a request across multiple microservices.

Tools:

- Spring Cloud Sleuth
 - Zipkin
-

22. What is Messaging in Microservices?**Answer:**

Messaging allows asynchronous communication between services.

Tools:

- Kafka
 - RabbitMQ
-

23. What is the difference between synchronous and asynchronous communication?

Synchronous	Asynchronous
Wait for response	No waiting
REST APIs	Messaging systems

24. What is Spring Data JPA?**Answer:**

Spring Data JPA simplifies database access by reducing boilerplate code for **CRUD operations**.

Example:

```
public interface UserRepository extends JpaRepository<User, Long> {  
}
```

25. What is the use of application.properties or application.yml?

Answer:

These files store **configuration settings** like:

- Server port
 - Database connection
 - API configuration
-

26. What is the difference between PUT and POST?

POST

PUT

Create new resource Update existing resource

Not idempotent Idempotent

27. What is Swagger?

Answer:

Swagger is a tool used to **document and test REST APIs**.

It provides a UI to test APIs.

Example:

- Swagger UI
 - OpenAPI
-

28. What is Exception Handling in Spring Boot?

Answer:

Spring Boot provides `@ControllerAdvice` and `@ExceptionHandler` to manage global exceptions.

Example:

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<?> handleException(){
        return ResponseEntity.status(500).body("Error occurred");
    }
}
```

29. What is API Versioning?

Answer:

API Versioning manages different versions of APIs.

Example:

/api/v1/users

30. What is the role of CI/CD in Microservices?

Answer:

CI/CD automates **build, testing, and deployment** of microservices.

Common tools:

- Jenkins
- GitLab CI
- GitHub Actions

Job Portal Microservices Application:

This project was developed to **simplify the job searching and recruitment process** by providing a platform where **job seekers can search and apply for jobs, and companies can post job openings and manage applications**.

The application was built using a **microservices architecture**, where different services handle different functionalities of the system.

The **frontend of the application was developed using and ReactJS**, which provides a responsive and user-friendly interface for users to browse jobs, manage profiles, and apply for positions.

For the **backend**, we used **Spring Boot** to develop **multiple microservices**, and **MySQL** was used to **manage the databases** for storing user information, job postings, applications, and profile details.

To enable **real-time notifications**, we used **Apache Kafka**, which allows services to communicate through event-based messaging.

Some key features include:

- **User Authentication and Authorization**, allowing users to register and log in securely.
- **Job Posting and Application Management**, where companies can post jobs and students can apply.
- **Profile and Resume Management**, allowing users to upload resumes and maintain their professional profiles.
- **Real-time Notifications**, which notify users when they apply for jobs or when companies update job status.
- **Admin Dashboard**, which helps administrators monitor users and platform activity.

The project was divided into **multiple microservices** such as **User Service, Job Service, Application Service, Profile Service, Resume Service, Notification Service, Admin Service, and Company Service**, where each service handles a specific functionality of the platform.

This project helped me gain practical experience in **Spring Boot microservices architecture, REST API development, database management using MySQL, and event-driven communication using Apache Kafka**, as well as **frontend-backend integration using React and Axios**.

Most Likely Interview Questions From Your Job Portal Project

1. Why did you use Microservices instead of Monolithic architecture?

They want to check whether you understand **microservices advantages**.

Expected points:

- Independent services
 - Better scalability
 - Easier maintenance
 - Fault isolation
-

2. How many microservices are in your project and what are their roles?

You should explain:

Microservice	Role
User Service	Authentication & user management
Job Service	Job posting and job listing
Application Service	Handling job applications
Profile Service	User profile information
Resume Service	Resume upload & storage
Notification Service	Sends notifications
Admin Service	Admin monitoring
Company Service	Company details

3. How do your microservices communicate with each other?

Expected answer:

- **REST APIs (Synchronous communication)**
 - **Apache Kafka (Asynchronous communication)**
-

4. Why did you use Apache Kafka in your project?

They want to check **event-driven architecture knowledge**.

Example answer:

Kafka is used for **real-time notifications**.

When a user applies for a job, the **Application Service publishes an event**, and the **Notification Service consumes the event** to send notifications.

5. Why did you use separate databases for each service?

Expected concept:

Database-per-service pattern

Benefits:

- Data isolation

- Independent scaling
 - Loose coupling
-

6. What happens when a user applies for a job?

You should explain the **flow**:

1. User logs in
 2. User selects job
 3. Application Service stores application
 4. Kafka event triggered
 5. Notification Service receives event
 6. Recruiter gets notification
-

7. What role does Spring Security play in your project?

Expected answer:

Spring Security is used for:

- Authentication
 - Authorization
 - Protecting APIs
-

8. How does the frontend communicate with backend?

Expected answer:

Frontend built using **React** communicates with backend **REST APIs** using **Axios**.

Example:

```
axios.get("/api/jobs")
```

9. What problems did you face while building this project?

Good answers:

- Managing multiple microservices
- Kafka configuration
- Database management

- API integration between React and Spring Boot
-

10. If one microservice fails, what happens?

They are testing **microservice fault tolerance**.

Expected idea:

Other services continue working because they are **independent**.

Example:

If **Notification Service fails**, job application still works.

11. How would you deploy this project in production?

Expected tools:

- Docker
 - Kubernetes
 - CI/CD pipeline
 - Cloud deployment
-

12. What improvements would you add in the future?

Good answers:

- JWT authentication
- Redis caching
- Docker containerization
- Kubernetes deployment
- Elasticsearch for job search

Role Based

My Role in the Project

In this project, I mainly worked on Profile Management Service and Resume Service, and I was also involved in integrating microservices and developing parts of the frontend using ReactJS.

For the Profile Management Service, I developed APIs that allow users to create, update, and view their profile information, such as personal details, skills, and experience. I implemented the backend using Spring Boot and MySQL, and designed REST APIs for managing profile data.

For the Resume Service, I implemented functionality that allows users to upload and manage their resumes. The service handles storing resume information and retrieving it when needed during the job application process.

I also worked on integrating different microservices using RestTemplate, which allows one service to communicate with another through REST APIs. This helped connect services like Application Service, Profile Service, and Resume Service.

On the frontend side, I contributed to building user interface components using ReactJS and used Axios to connect the frontend with backend APIs for features like profile management and resume upload.

Through this project, I gained practical experience in Spring Boot microservices development, REST API integration, database management, and frontend-backend communication.

1. What APIs did you create in the Profile Management Service?

Good Answer:

In the Profile Service I created REST APIs to manage user profile information.

Example APIs:

API	Description
GET /api/profiles/{userId}	Fetch user profile
POST /api/profiles	Create profile
PUT /api/profiles/{userId}	Update profile
DELETE /api/profiles/{userId}	Delete profile

These APIs allow users to create, view, and update their profile information such as skills, education, and experience.

2. How does Resume Upload work in your Resume Service?

Answer:

In the Resume Service, users can upload their resume files.

Process:

1. User uploads resume from frontend.
2. Request is sent to Resume Service.
3. The file is received using MultipartFile in Spring Boot.
4. Resume information is stored in the database.
5. Resume can later be fetched when applying for jobs.

Example controller method:

```
@PostMapping("/upload")
public ResponseEntity<String> uploadResume(@RequestParam MultipartFile file) {
    return ResponseEntity.ok("Resume uploaded successfully");
}
```

3. Why did you use RestTemplate?

Answer:

RestTemplate is used for communication between microservices.

For example:

When a user applies for a job, the Application Service may need profile details, so it calls the Profile Service API using RestTemplate.

Example:

```
RestTemplate restTemplate = new RestTemplate();
Profile profile = restTemplate.getForObject(
    "http://profile-service/api/profiles/" + userId,
    Profile.class
);
```

This allows services to fetch data from other services using REST APIs.

4. Why did you choose RestTemplate instead of Feign Client?

Good answer:

RestTemplate is simple and easy to use for basic REST communication.

However, in large microservice systems Feign Client is better because:

- Less boilerplate code

- Automatic service discovery integration
- Better maintainability

Feign is often used in Spring Cloud microservices.

5. What React components did you build?

Answer example:

I worked on frontend components related to:

- Profile creation form
- Profile update page
- Resume upload interface

These components used Axios to send API requests to the backend services.

Example:

```
axios.post("/api/resumes/upload", formData)
```

6. How did you connect React frontend with Spring Boot backend?

Answer:

React communicates with backend using REST APIs.

Process:

1. User interacts with UI
2. React sends API request using Axios
3. Spring Boot microservice processes request
4. Response is returned as JSON

Example:

```
axios.get("/api/profiles/" + userId)  
.then(response => setProfile(response.data))
```

7. What database tables did you create for Profile Service?

Example answer:

Profile table includes fields like:

- profile_id

- user_id
- name
- email
- skills
- education
- experience

This table stores all profile-related information for job seekers.

8. What challenges did you face while building these services?

Good answers:

- Handling file uploads in Resume Service
 - Integrating microservices using RestTemplate
 - Connecting React frontend with backend APIs
 - Managing multiple databases for different services
-

9. How would you improve your services in future?

Answer:

Future improvements could include:

- Using Feign Client instead of RestTemplate
 - Adding JWT authentication
 - Storing resumes in cloud storage (AWS S3)
 - Adding API Gateway for routing requests
-

10. How did you test your APIs?

Answer:

I tested APIs using:

- Postman
- Browser requests
- Frontend integration testing

Bus Booking System Project Explanation

Bus Booking System:

This project was developed to simplify the process of searching and booking bus tickets online. The system allows users to search buses based on source and destination, view available buses, and book or cancel tickets through a web interface.

The application was developed using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js.

The frontend of the application was built using React.js, which provides a responsive and interactive user interface. We also used Tailwind CSS to design modern and responsive UI components.

For the backend, we used Node.js and Express.js to create REST APIs that handle server-side logic such as user authentication, bus search, booking, and cancellation operations.

The system uses MongoDB as a NoSQL database to store user information, bus details, and booking records.

To ensure secure access, we implemented JWT (JSON Web Token) authentication, which allows users to securely register, log in, and access protected routes within the system.

Some key features include:

- User Registration and Login with JWT authentication
- Search buses based on source and destination
- View available buses and seat information
- Book bus tickets and cancel bookings
- Secure storage of booking and user data in MongoDB

The backend APIs were tested using Postman, and the frontend communicates with backend services using REST APIs.

This project helped me gain practical experience in full-stack web development using the MERN stack, API development using Express.js, authentication using JWT, and frontend-backend integration using React.js.

Follow-up Questions Interviewers May Ask

1. Why did you use the MERN stack?

Answer idea:

The MERN stack allows full-stack development using JavaScript for both frontend and backend.

Benefits:

- Single language (JavaScript)
 - Fast development
 - Scalable architecture
 - Easy frontend-backend integration
-

2. How does JWT authentication work in your project?

Expected explanation:

1. User logs in with credentials
 2. Server verifies user
 3. Server generates JWT token
 4. Token is sent to frontend
 5. Frontend stores token and sends it in future requests
 6. Backend verifies token to allow access
-

3. What APIs did you create in the Bus Booking System?

Example APIs:

API	Description
POST /api/auth/register	Register user
POST /api/auth/login	User login
GET /api/buses	View available buses
POST /api/bookings	Book ticket
DELETE /api/bookings/:id	Cancel booking

4. What collections did you create in MongoDB?

Example collections:

- Users
- Buses

- Bookings

These collections store user details, bus information, and booking records.

5. How does the bus booking flow work?

Flow:

1. User logs into system
 2. User searches buses using source and destination
 3. Available buses are displayed
 4. User selects bus and books ticket
 5. Booking information is stored in MongoDB
-

6. How does React communicate with the backend?

React communicates with backend APIs using HTTP requests.

Example:

```
axios.get("/api/buses")
```

The backend returns JSON data, which is displayed in the UI